

Bài toán tối ưu

1 Mở đầu

Trong nhiều bài toán thực tế, ta không chỉ cần tìm một phương án thoả mãn điều kiện, mà còn cần tìm phương án tốt nhất theo một tiêu chí nào đó. Những bài toán như vậy được gọi là bài toán tối ưu.

Dạng tổng quát của bài toán tối ưu tổ hợp là:

$$f(x) \rightarrow \max \quad \text{hoặc} \quad f(x) \rightarrow \min$$

với điều kiện

$$x \in D,$$

trong đó:

- D là tập các phương án khả thi;
- $f(x)$ là hàm mục tiêu;
- phương án $x^* \in D$ làm cho $f(x)$ đạt lớn nhất hoặc nhỏ nhất được gọi là phương án tối ưu.

Trong tài liệu này, ta sẽ giải thích 4 bài toán tối ưu quen thuộc:

1. Bài toán cái túi;
2. Bài toán người du lịch;
3. Bài toán cho thuê máy;
4. Bài toán phân công công việc.

Sau đó, tài liệu trình bày hai hướng tiếp cận cơ bản:

- thuật toán duyệt toàn bộ (Brute Force or Exhaustive);
- thuật toán nhánh cận (Branch and Bound).

Minh hoạ xem thêm tại file **Optimization.html**.

1.1 Ý tưởng chung của phương pháp nhánh cận

Phương pháp nhánh cận là phương pháp cải tiến từ duyệt toàn bộ. Thay vì xét hết mọi phương án, ta chia không gian nghiệm thành các nhánh và dùng một cận để loại bỏ sớm những nhánh chắc chắn không thể chứa nghiệm tối ưu.

Tư tưởng chung gồm hai bước:

- **Phân nhánh (Branch):** phân nhánh từ một phương án bộ phận thành các phương án bộ phận sâu hơn;
- **Cận (Bound):** tính giá trị cận cho phương án bộ phận để quyết định có nên xét tiếp nhánh đó hay không.

Giả sử đang giải bài toán cực đại. Với một phương án bộ phận u , nếu ta tính được cận trên $g(u)$ sao cho

$$f(x) \leq g(u) \quad \text{với mọi nghiệm đầy đủ } x \text{ sinh ra từ } u,$$

thì:

- nếu $g(u) \leq FOPT$, trong đó $FOPT$ là giá trị tốt nhất hiện có, ta loại nhánh đó;
- nếu $g(u) > FOPT$, ta tiếp tục phân nhánh.

Ngược lại, nếu là bài toán cực tiểu, ta dùng cận dưới:

$$g(u) \leq f(x),$$

và nếu

$$g(u) \geq FOPT,$$

thì có thể loại nhánh đó.

2 Bài toán cái túi (Knapsack Problem)

2.1 Phát biểu bài toán

Có một cái túi chỉ mang được khối lượng tối đa B . Có N đồ vật, đồ vật thứ i có:

- trọng lượng a_i ;
- giá trị sử dụng c_i .

Mỗi đồ vật hoặc được chọn, hoặc không được chọn. Ta cần chọn một số đồ vật sao cho:

- tổng trọng lượng không vượt quá B ;
- tổng giá trị là lớn nhất.

2.2 Mô hình toán học

Gọi

$$x_i = \begin{cases} 1, & \text{nếu chọn đồ vật } i, \\ 0, & \text{nếu không chọn đồ vật } i. \end{cases}$$

Khi đó:

$$a_1x_1 + a_2x_2 + \cdots + a_Nx_N \leq B$$

và hàm mục tiêu là:

$$f(x) = c_1x_1 + c_2x_2 + \cdots + c_Nx_N \rightarrow \max.$$

2.3 Ví dụ

Giả sử có 4 đồ vật như sau:

Đồ vật	Trọng lượng	Giá trị
1	2	3
2	3	4
3	4	5
4	5	6

Sức chứa của túi là:

$$B = 5.$$

2.4 Bài toán cái túi bằng duyệt toàn bộ

Vì có 4 đồ vật nên mỗi phương án là một xâu nhị phân dài 4:

$$(x_1, x_2, x_3, x_4).$$

Ta xét tất cả các phương án:

Phương án	Tổng trọng lượng	Tổng giá trị	Hợp lệ?
(0, 0, 0, 0)	0	0	Có
(1, 0, 0, 0)	2	3	Có
(0, 1, 0, 0)	3	4	Có
(0, 0, 1, 0)	4	5	Có
(0, 0, 0, 1)	5	6	Có
(1, 1, 0, 0)	5	7	Có
(1, 0, 1, 0)	6	8	Không
(1, 0, 0, 1)	7	9	Không
(0, 1, 1, 0)	7	9	Không
(0, 1, 0, 1)	8	10	Không
(0, 0, 1, 1)	9	11	Không
(1, 1, 1, 0)	9	12	Không
(1, 1, 0, 1)	10	13	Không
(1, 0, 1, 1)	11	14	Không
(0, 1, 1, 1)	12	15	Không
(1, 1, 1, 1)	14	18	Không

Trong các phương án hợp lệ, giá trị lớn nhất là 7, đạt được tại phương án:

$$(x_1, x_2, x_3, x_4) = (1, 1, 0, 0).$$

Phương án tối ưu là chọn đồ vật 1 và đồ vật 2:

$$x^* = (1, 1, 0, 0)$$

với

$$f(x^*) = 7.$$

Với mỗi đồ vật, ta có 2 lựa chọn: chọn hoặc không chọn. Do đó, với N đồ vật, ta có tất cả 2^N phương án. Ta duyệt toàn bộ các phương án, kiểm tra phương án nào thoả mãn điều kiện trọng lượng, rồi chọn phương án có tổng giá trị lớn nhất.

Dữ liệu vào:

- N : số đồ vật;
- $a[1..N]$: trọng lượng các đồ vật;
- $c[1..N]$: giá trị các đồ vật;
- B : sức chứa của túi.

Dữ liệu ra:

- phương án tối ưu X^* ;

- giá trị tối ưu $FOPT$.

Thuật toán giả:

Bước 1. Khởi tạo $FOPT = -\infty$, $XOPT = \emptyset$.

Bước 2. Duyệt tất cả các vectơ nhị phân $X = (x_1, x_2, \dots, x_N)$.

Bước 3. Với mỗi phương án X , tính:

$$W = a_1x_1 + a_2x_2 + \dots + a_Nx_N, \quad V = c_1x_1 + c_2x_2 + \dots + c_Nx_N.$$

Bước 4. Nếu $W \leq B$ và $V > FOPT$ thì cập nhật:

$$FOPT = V, \quad XOPT = X.$$

Bước 5. Trả về $(XOPT, FOPT)$.

Độ phức tạp của cách duyệt toàn bộ là cỡ 2^N vì có 2^N phương án cần xét.

2.5 Bài toán cái túi bằng phương pháp nhánh cận

2.5.1 Phương án bộ phận

Ở mức thứ k , ta đã quyết định các biến:

$$x_1, x_2, \dots, x_k.$$

Đây là một phương án bộ phận. Các biến còn lại $x_{k+1}, \dots, x_N$ chưa được quyết định.

Ký hiệu:

$$W_k = \sum_{i=1}^k a_i x_i, \quad V_k = \sum_{i=1}^k c_i x_i.$$

2.5.2 Xây dựng cận trên

Ta sắp xếp các đồ vật theo tỉ số giá trị trên trọng lượng giảm dần:

$$\frac{c_1}{a_1} \geq \frac{c_2}{a_2} \geq \dots \geq \frac{c_N}{a_N}.$$

Với một phương án bộ phận ở mức k , phần sức chứa còn lại là:

$$R_k = B - W_k.$$

Một cận trên đơn giản là giả sử ta có thể nhét tiếp đồ vật theo kiểu “phân số”. Khi đó:

$$g(x_1, \dots, x_k) = V_k + \text{giá trị lớn nhất có thể thêm vào từ phần còn lại.}$$

Trong trình bày sơ cấp, có thể viết gọn gần đúng dưới dạng:

$$g(x_1, \dots, x_k) = V_k + R_k \cdot \frac{c_{k+1}}{a_{k+1}},$$

với giả thiết các đồ vật đã được sắp theo tỉ số giảm dần. Nếu muốn cận chặt hơn, ta lấp dần phần sức chứa còn lại bằng các đồ vật tiếp theo theo thứ tự tỉ số giảm dần, và có thể lấy phân số ở đồ vật cuối cùng.

Đây là cận trên vì nó đánh giá lạc quan hơn giá trị thật sự có thể đạt được.

2.5.3 Điều kiện loại nhánh

- Nếu $W_k > B$ thì phương án bộ phận không hợp lệ, loại ngay.
- Nếu

$$g(x_1, \dots, x_k) \leq FOPT,$$

thì nhánh này không thể cho nghiệm tốt hơn nghiệm tốt nhất hiện tại, nên loại.

2.5.4 Thuật toán nhánh cận

Dữ liệu vào:

- N : số đồ vật;
- a_i : trọng lượng;
- c_i : giá trị;
- B : sức chứa túi.

Dữ liệu ra:

- phương án tối ưu X^* ;
- giá trị tối ưu $FOPT$.

Thuật toán:

Bước 1. Sắp xếp các đồ vật theo tỉ số $\frac{c_i}{a_i}$ giảm dần.

Bước 2. Khởi tạo:

$$FOPT = 0, \quad XOPT = \emptyset.$$

Bước 3. Tại mức k , thử hai khả năng: $x_k = 1$ hoặc $x_k = 0$.

Bước 4. Với mỗi phương án bộ phận:

- tính W_k, V_k ;
- nếu $W_k > B$ thì loại;

- tính cận trên $g(x_1, \dots, x_k)$;
- nếu $g(x_1, \dots, x_k) \leq FOPT$ thì loại;
- nếu $k = N$ và $V_k > FOPT$ thì cập nhật nghiệm tốt nhất.

Bước 5. Kết thúc khi không còn nhánh nào cần xét.

Ưu điểm của nhánh cận là không cần xét hết 2^N phương án. Nếu cận trên đủ tốt, rất nhiều nhánh sẽ bị loại sớm.

2.6 Minh họa chi tiết phương pháp nhánh cận cho bài toán cái túi

Xét lại ví dụ đã nêu ở trên:

$$f(x) = 3x_1 + 4x_2 + 5x_3 + 6x_4 \rightarrow \max$$

với điều kiện

$$2x_1 + 3x_2 + 4x_3 + 5x_4 \leq 5, \quad x_i \in \{0, 1\}.$$

2.6.1 Bước 1. Sắp xếp theo tỉ số giá trị trên trọng lượng

Ta tính:

$$\begin{aligned} \frac{c_1}{a_1} &= \frac{3}{2} = 1.5, & \frac{c_2}{a_2} &= \frac{4}{3} \approx 1.33, \\ \frac{c_3}{a_3} &= \frac{5}{4} = 1.25, & \frac{c_4}{a_4} &= \frac{6}{5} = 1.2. \end{aligned}$$

Do đó:

$$\frac{c_1}{a_1} > \frac{c_2}{a_2} > \frac{c_3}{a_3} > \frac{c_4}{a_4}.$$

Như vậy, thứ tự xét các biến là:

$$x_1, x_2, x_3, x_4.$$

2.6.2 Bước 2. Khởi tạo

Ban đầu, chưa chọn đồ vật nào:

$$W = 0, \quad V = 0.$$

Ta đặt:

$$FOPT = 0.$$

Ở đây:

- W là tổng trọng lượng hiện tại;
- V là tổng giá trị hiện tại;
- $FOPT$ là giá trị tốt nhất đã tìm được cho đến thời điểm hiện tại.

2.6.3 Bước 3. Tính cận trên tại nút gốc

Tại nút gốc, sức chứa còn lại là:

$$R = 5.$$

Để tính cận trên, ta giả sử có thể nhét các đồ vật còn lại theo kiểu phân số.

- Lấy trọn đồ vật 1: trọng lượng 2, giá trị 3.
- Còn lại sức chứa 3.
- Lấy trọn đồ vật 2: trọng lượng 3, giá trị 4.

Vậy cận trên tại nút gốc là:

$$UB = 3 + 4 = 7.$$

Do

$$UB = 7 > FOPT = 0,$$

nên ta tiếp tục phân nhánh.

2.6.4 Bước 4. Xét nhánh $x_1 = 1$

Ta thử chọn đồ vật 1:

$$x_1 = 1.$$

Khi đó:

$$W = 2, \quad V = 3.$$

Sức chứa còn lại:

$$R = 5 - 2 = 3.$$

Tính cận trên:

- Đồ vật tiếp theo là đồ vật 2, có trọng lượng 3, giá trị 4.
- Ta có thể lấy trọn đồ vật 2.

Do đó:

$$UB = 3 + 4 = 7.$$

Vì

$$UB = 7 > FOPT = 0,$$

nên ta tiếp tục xét sâu hơn.

Nhánh con: $x_1 = 1, x_2 = 1$ Nếu chọn tiếp đồ vật 2, ta có:

$$W = 2 + 3 = 5, \quad V = 3 + 4 = 7.$$

Đây là một phương án khả thi vì:

$$W = 5 \leq 5.$$

Hơn nữa, túi đã đầy nên không cần xét thêm. Ta cập nhật:

$$FOPT = 7.$$

Phương án tốt nhất hiện tại là:

$$(1, 1, 0, 0).$$

Nhánh con: $x_1 = 1, x_2 = 0$ Bây giờ xét trường hợp chọn đồ vật 1 nhưng không chọn đồ vật 2:

$$x_1 = 1, \quad x_2 = 0.$$

Khi đó:

$$W = 2, \quad V = 3.$$

Sức chứa còn lại:

$$R = 3.$$

Ta tính cận trên bằng cách nhét phân số từ đồ vật tiếp theo là đồ vật 3:

- đồ vật 3 có trọng lượng 4, giá trị 5;
- chỉ còn chỗ 3, nên tối đa chỉ lấy được $\frac{3}{4}$ đồ vật 3.

Giá trị tối đa có thể thêm vào là:

$$\frac{3}{4} \cdot 5 = 3.75.$$

Do đó:

$$UB = 3 + 3.75 = 6.75.$$

So sánh với nghiệm tốt nhất hiện tại:

$$UB = 6.75 < FOPT = 7.$$

Vì vậy, nhánh này bị loại. Ta không cần xét tiếp x_3 và x_4 .

2.6.5 Bước 5. Xét nhánh $x_1 = 0$

Ta xét trường hợp không chọn đồ vật 1:

$$x_1 = 0.$$

Khi đó:

$$W = 0, \quad V = 0.$$

Sức chứa còn lại:

$$R = 5.$$

Ta tính cận trên:

- lấy trọn đồ vật 2: được giá trị 4, còn sức chứa 2;
- đồ vật 3 có trọng lượng 4, giá trị 5, nên chỉ lấy được $\frac{2}{4}$.

Giá trị thêm tối đa từ đồ vật 3 là:

$$\frac{2}{4} \cdot 5 = 2.5.$$

Do đó:

$$UB = 4 + 2.5 = 6.5.$$

So sánh với nghiệm tốt nhất hiện tại:

$$UB = 6.5 < FOPT = 7.$$

Vì vậy, nhánh này cũng bị loại. Không cần xét tiếp x_2, x_3, x_4 .

2.6.6 Bước 6. Kết luận

Sau khi xét các nhánh cần thiết và loại bỏ các nhánh không thể tốt hơn nghiệm hiện tại, ta thu được phương án tối ưu:

$$x^* = (1, 1, 0, 0)$$

với giá trị tối ưu:

$$f(x^*) = 7.$$

Nói cách khác, ta chọn:

- đồ vật 1;
- đồ vật 2.

Tổng trọng lượng là:

$$2 + 3 = 5,$$

và tổng giá trị là:

$$3 + 4 = 7.$$

2.6.7 Tóm tắt cây nhánh cận

Ta có thể tóm tắt quá trình trên bằng sơ đồ ngắn gọn sau:

$$() : \quad V = 0, \quad W = 0, \quad UB = 7$$

Từ nút gốc, tách thành hai nhánh:

$$(1) : \quad V = 3, \quad W = 2, \quad UB = 7$$

$$(0) : \quad V = 0, \quad W = 0, \quad UB = 6.5 \Rightarrow \text{loại}$$

Từ nhánh (1), tiếp tục tách:

$$(1, 1) : \quad V = 7, \quad W = 5 \Rightarrow \text{nghiệm khả thi, cập nhật } FOPT = 7$$

$$(1, 0) : \quad V = 3, \quad W = 2, \quad UB = 6.75 \Rightarrow \text{loại}$$

Do đó, không cần xét thêm nhánh nào nữa. Ví dụ này cho thấy phương pháp nhánh cận giúp giảm số phương án cần xét. Thay vì duyệt cả $2^4 = 16$ phương án, ta chỉ cần đi sâu vào một số rất ít nhánh rồi loại bỏ sớm các nhánh còn lại nhờ cận trên.

2.7 Giải ví dụ bài toán cái túi bằng quy hoạch động (Dynamic Programming)

2.7.1 Ý tưởng quy hoạch động

Đặt

$$dp[i][w]$$

là giá trị lớn nhất có thể đạt được khi chỉ xét i đồ vật đầu tiên và sức chứa của túi là w . Khi đó:

- nếu không chọn đồ vật i , ta có:

$$dp[i][w] = dp[i-1][w];$$

- nếu chọn được đồ vật i (nghĩa là $a_i \leq w$), ta có:

$$dp[i][w] = \max\{dp[i-1][w], dp[i-1][w - a_i] + c_i\}.$$

2.7.2 Bước 1. Khởi tạo

Nếu chưa xét đồ vật nào thì giá trị lớn nhất luôn bằng 0:

$$dp[0][w] = 0, \quad \forall w = 0, 1, 2, 3, 4, 5.$$

2.7.3 Bước 2. Xét đồ vật 1

Đồ vật 1 có:

$$a_1 = 2, \quad c_1 = 3.$$

Ta tính:

- $dp[1][0] = 0;$

- $dp[1][1] = 0$ vì chưa đủ sức chứa để chọn đồ vật 1;
- $dp[1][2] = 3$;
- $dp[1][3] = 3$;
- $dp[1][4] = 3$;
- $dp[1][5] = 3$.

Vậy hàng thứ nhất là:

$$0, 0, 3, 3, 3, 3.$$

2.7.4 Bước 3. Xét đồ vật 2

Đồ vật 2 có:

$$a_2 = 3, \quad c_2 = 4.$$

Ta tính:

- $dp[2][0] = 0$;
- $dp[2][1] = 0$;
- $dp[2][2] = dp[1][2] = 3$;
- với $w = 3$:

$$dp[2][3] = \max\{dp[1][3], dp[1][0] + 4\} = \max\{3, 4\} = 4;$$

- với $w = 4$:

$$dp[2][4] = \max\{dp[1][4], dp[1][1] + 4\} = \max\{3, 4\} = 4;$$

- với $w = 5$:

$$dp[2][5] = \max\{dp[1][5], dp[1][2] + 4\} = \max\{3, 7\} = 7.$$

Vậy hàng thứ hai là:

$$0, 0, 3, 4, 4, 7.$$

2.7.5 Bước 4. Xét đồ vật 3

Đồ vật 3 có:

$$a_3 = 4, \quad c_3 = 5.$$

Ta tính:

- $dp[3][0] = 0$;
- $dp[3][1] = 0$;

- $dp[3][2] = 3$;
- $dp[3][3] = 4$;
- với $w = 4$:

$$dp[3][4] = \max\{dp[2][4], dp[2][0] + 5\} = \max\{4, 5\} = 5;$$

- với $w = 5$:

$$dp[3][5] = \max\{dp[2][5], dp[2][1] + 5\} = \max\{7, 5\} = 7.$$

Vậy hàng thứ ba là:

$$0, 0, 3, 4, 5, 7.$$

2.7.6 Bước 5. Xét đồ vật 4

Đồ vật 4 có:

$$a_4 = 5, \quad c_4 = 6.$$

Ta tính:

- $dp[4][0] = 0$;
- $dp[4][1] = 0$;
- $dp[4][2] = 3$;
- $dp[4][3] = 4$;
- $dp[4][4] = 5$;
- với $w = 5$:

$$dp[4][5] = \max\{dp[3][5], dp[3][0] + 6\} = \max\{7, 6\} = 7.$$

Vậy hàng thứ tư là:

$$0, 0, 3, 4, 5, 7.$$

2.7.7 Bảng quy hoạch động

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Từ bảng trên, giá trị tối ưu là:

$$dp[4][5] = 7.$$

2.7.8 Truy vết phương án tối ưu

Ta truy ngược từ ô $dp[4][5]$:

- $dp[4][5] = 7 = dp[3][5]$, nên không chọn đồ vật 4;
- $dp[3][5] = 7 = dp[2][5]$, nên không chọn đồ vật 3;
- $dp[2][5] = 7 \neq dp[1][5] = 3$, nên có chọn đồ vật 2.

Khi chọn đồ vật 2, ta giảm sức chứa:

$$w = 5 - 3 = 2.$$

Tiếp tục xét:

- $dp[1][2] = 3 \neq dp[0][2] = 0$, nên có chọn đồ vật 1.

Vậy phương án tối ưu là:

$$x^* = (1, 1, 0, 0),$$

nghĩa là chọn đồ vật 1 và đồ vật 2.

Tổng trọng lượng là:

$$2 + 3 = 5,$$

và tổng giá trị là:

$$3 + 4 = 7.$$

Đối với bài toán cái túi, quy hoạch động thường là phương pháp rất hiệu quả khi sức chứa B không quá lớn. Trong ví dụ này, quy hoạch động cho cùng nghiệm tối ưu như phương pháp vét cạn và phương pháp nhánh cận:

$$x^* = (1, 1, 0, 0), \quad f(x^*) = 7.$$

3 Bài toán người du lịch (Travelling Salesman Problem)

3.1 Phát biểu bài toán

Có N thành phố. Một người du lịch muốn:

- xuất phát từ một thành phố;
- đi qua mỗi thành phố đúng một lần;
- rồi quay lại điểm xuất phát.

Biết chi phí đi từ thành phố i đến thành phố j là c_{ij} . Cần tìm hành trình có tổng chi phí nhỏ nhất.

Mỗi phương án là một thứ tự đi qua các thành phố. Ta cần so sánh tổng chi phí của các hành trình hợp lệ và chọn hành trình có chi phí nhỏ nhất.

Ghi chú. Trong nhiều tài liệu, bài toán người du lịch được xét dưới dạng đặc biệt gọi là *metric TSP*. Khi đó, chi phí di chuyển giữa hai thành phố được ký hiệu bởi $d(i, j)$ và thoả mãn hai tính chất:

$$d(i, j) = d(j, i)$$

và

$$d(i, k) \leq d(i, j) + d(j, k),$$

với mọi thành phố i, j, k .

Tính chất thứ nhất nói rằng chi phí đi từ i đến j bằng chi phí đi từ j đến i . Tính chất thứ hai là bất đẳng thức tam giác, nghĩa là đi trực tiếp từ i đến k không đắt hơn đi vòng qua một thành phố trung gian j .

Tuy nhiên, trong slides, ta đôi khi gặp bài toán người du lịch dưới dạng tổng quát hơn, chỉ giả sử đã biết ma trận chi phí $C = (c_{ij})$. Vì vậy, không nhất thiết phải giả thiết trước rằng

$$c_{ij} = c_{ji}$$

hay

$$c_{ik} \leq c_{ij} + c_{jk}.$$

3.2 Ví dụ

Xét 4 thành phố: 1, 2, 3, 4.

Ma trận chi phí:

$$C = \begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 22 & 25 \\ 15 & 22 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix}.$$

Ta cố định điểm xuất phát là thành phố 1. Khi đó chỉ cần xét các hoán vị của 2, 3, 4. Việc cố định thành phố xuất phát giúp tránh đếm lặp các chu trình chỉ khác nhau ở vị trí bắt đầu.

3.3 Giải bằng duyệt toàn bộ

Các hành trình có thể có:

$$1. 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$$

$$10 + 22 + 30 + 20 = 82$$

$$2. 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

$$10 + 25 + 30 + 15 = 80$$

$$3. 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$$

$$15 + 22 + 25 + 20 = 82$$

$$4. 1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

$$15 + 30 + 25 + 10 = 80$$

$$5. 1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1$$

$$20 + 25 + 22 + 15 = 82$$

$$6. 1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$$

$$20 + 30 + 22 + 10 = 82$$

Từ đó, chi phí nhỏ nhất là: 80

Có hai hành trình tối ưu:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

hoặc

$$1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1.$$

3.4 Minh họa chi tiết phương pháp nhánh cận cho bài toán người du lịch

Xét lại ví dụ đã nêu ở trên với 4 thành phố 1, 2, 3, 4 và ma trận chi phí:

$$C = \begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 22 & 25 \\ 15 & 22 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix}.$$

Ta cố định thành phố xuất phát là thành phố 1. Khi đó, ta cần tìm chu trình ngắn nhất dạng:

$$1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow 1,$$

trong đó mỗi thành phố 2, 3, 4 xuất hiện đúng một lần.

3.4.1 Bước 1. Tính chi phí nhỏ nhất $c_{\min}$

Ta xét các phần tử ngoài đường chéo chính của ma trận chi phí. Giá trị nhỏ nhất là:

$$c_{\min} = 10.$$

Giá trị này sẽ được dùng để xây dựng cận dưới.

3.4.2 Bước 2. Khởi tạo tại nút gốc

Ban đầu, ta mới đứng ở thành phố xuất phát:

$$x_1 = 1.$$

Chi phí hiện tại là:

$$\delta_1 = 0.$$

Vì còn phải đi qua 3 thành phố chưa thăm và quay lại điểm xuất phát, nên còn ít nhất

$$N - k + 1 = 4 - 1 + 1 = 4$$

cung đường nữa.

Do đó, cận dưới tại nút gốc là:

$$g(1) = \delta_1 + 4c_{\min} = 0 + 4 \cdot 10 = 40.$$

Ban đầu ta đặt:

$$FOPT = +\infty.$$

Vì

$$g(1) = 40 < FOPT,$$

nên ta tiếp tục phân nhánh.

3.4.3 Bước 3. Phân nhánh từ thành phố 1

Từ thành phố 1, ta có thể đi tiếp đến một trong ba thành phố:

$$2, 3, 4.$$

Nhánh 1: $1 \rightarrow 2$ Khi đó:

$$\delta_2 = c_{1,2} = 10.$$

Cận dưới là:

$$g(1, 2) = 10 + 3 \cdot 10 = 40.$$

Nhánh 2: $1 \rightarrow 3$ Khi đó:

$$\delta_2 = c_{1,3} = 15.$$

Cận dưới là:

$$g(1, 3) = 15 + 3 \cdot 10 = 45.$$

Nhánh 3: $1 \rightarrow 4$ Khi đó:

$$\delta_2 = c_{1,4} = 20.$$

Cận dưới là:

$$g(1, 4) = 20 + 3 \cdot 10 = 50.$$

Hiện tại chưa có nghiệm đầy đủ nào, nên chưa loại nhánh nào. Ta tiếp tục xét sâu hơn.

3.4.4 Bước 4. Xét tiếp các nhánh con

Từ nhánh $1 \rightarrow 2$ Ta còn hai khả năng.

Nhánh $1 \rightarrow 2 \rightarrow 3$

Khi đó:

$$\delta_3 = c_{1,2} + c_{2,3} = 10 + 22 = 32.$$

Cận dưới:

$$g(1, 2, 3) = 32 + 2 \cdot 10 = 52.$$

Đi tiếp bắt buộc đến thành phố 4, ta được chu trình đầy đủ:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1.$$

Chi phí:

$$10 + 22 + 30 + 20 = 82.$$

Do đó cập nhật:

$$FOPT = 82.$$

Nhánh $1 \rightarrow 2 \rightarrow 4$

Khi đó:

$$\delta_3 = c_{1,2} + c_{2,4} = 10 + 25 = 35.$$

Cận dưới:

$$g(1, 2, 4) = 35 + 2 \cdot 10 = 55.$$

Vì

$$55 < 82,$$

nên ta tiếp tục xét.

Đi tiếp bắt buộc đến thành phố 3, ta được chu trình:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1.$$

Chi phí:

$$10 + 25 + 30 + 15 = 80.$$

Vì

$$80 < 82,$$

nên cập nhật:

$$FOPT = 80.$$

Từ nhánh $1 \rightarrow 3$ Ta còn hai khả năng.

Nhánh $1 \rightarrow 3 \rightarrow 2$

Khi đó:

$$\delta_3 = c_{1,3} + c_{3,2} = 15 + 22 = 37.$$

Cận dưới:

$$g(1, 3, 2) = 37 + 2 \cdot 10 = 57.$$

Vì

$$57 < 80,$$

nên chưa thể loại nhánh này.

Đi tiếp bắt buộc đến thành phố 4, ta được chu trình:

$$1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1.$$

Chi phí:

$$15 + 22 + 25 + 20 = 82.$$

Không cập nhật vì

$$82 > 80.$$

Nhánh $1 \rightarrow 3 \rightarrow 4$

Khi đó:

$$\delta_3 = c_{1,3} + c_{3,4} = 15 + 30 = 45.$$

Cận dưới:

$$g(1, 3, 4) = 45 + 2 \cdot 10 = 65.$$

Vì

$$65 < 80,$$

nên tiếp tục xét.

Đi tiếp bắt buộc đến thành phố 2, ta được chu trình:

$$1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1.$$

Chi phí:

$$15 + 30 + 25 + 10 = 80.$$

Chi phí này bằng nghiệm tốt nhất hiện tại, nên ta có thêm một hành trình tối ưu nữa.

Từ nhánh $1 \rightarrow 4$ Ta còn hai khả năng.

Nhánh $1 \rightarrow 4 \rightarrow 2$

Khi đó:

$$\delta_3 = c_{1,4} + c_{4,2} = 20 + 25 = 45.$$

Cận dưới:

$$g(1, 4, 2) = 45 + 2 \cdot 10 = 65.$$

Vì

$$65 < 80,$$

nên chưa loại nhánh này.

Đi tiếp bắt buộc đến thành phố 3, ta được chu trình:

$$1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1.$$

Chi phí:

$$20 + 25 + 22 + 15 = 82.$$

Không cập nhật.

Nhánh $1 \rightarrow 4 \rightarrow 3$

Khi đó:

$$\delta_3 = c_{1,4} + c_{4,3} = 20 + 30 = 50.$$

Cận dưới:

$$g(1, 4, 3) = 50 + 2 \cdot 10 = 70.$$

Vì

$$70 < 80,$$

nên chưa loại nhánh này.

Đi tiếp bắt buộc đến thành phố 2, ta được chu trình:

$$1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1.$$

Chi phí:

$$20 + 30 + 22 + 10 = 82.$$

Không cập nhật.

3.4.5 Bước 5. Kết luận

Sau khi xét các nhánh, ta thu được chi phí nhỏ nhất là:

$$FOPT = 80.$$

Có hai hành trình tối ưu:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

và

$$1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1.$$

3.5 Minh họa thuật toán xấp xỉ dựa trên cây khung nhỏ nhất cho bài toán người du lịch

Xét lại ví dụ với 4 thành phố 1, 2, 3, 4 và ma trận chi phí:

$$C = \begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 22 & 25 \\ 15 & 22 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix}.$$

Ta nhận thấy ma trận này đối xứng:

$$c_{ij} = c_{ji},$$

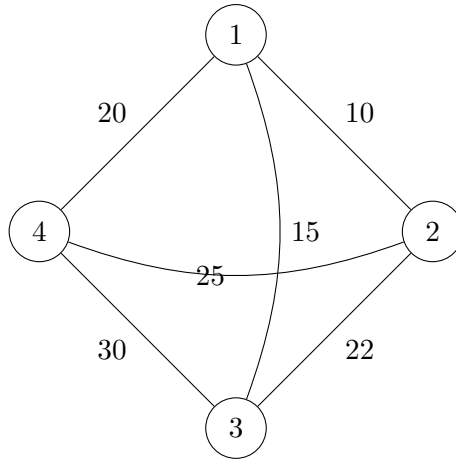
và các chi phí cũng thoả mãn bất đẳng thức tam giác. Vì vậy, ta có thể áp dụng **thuật toán xấp xỉ** dựa trên cây khung nhỏ nhất. Ở đây là **2-approximation algorithm**.

3.5.1 Bước 1. Xây dựng đồ thị đầy đủ có trọng số

Mỗi thành phố được biểu diễn bởi một đỉnh. Trọng số cạnh nối giữa hai đỉnh là chi phí đi giữa hai thành phố tương ứng.

Các cạnh của đồ thị là:

$$\begin{aligned} (1, 2) &= 10, & (1, 3) &= 15, & (1, 4) &= 20, \\ (2, 3) &= 22, & (2, 4) &= 25, & (3, 4) &= 30. \end{aligned}$$



3.5.2 Bước 2. Tìm cây khung nhỏ nhất (Minimum Spanning Tree)

Ta chọn các cạnh có trọng số nhỏ nhất sao cho nối được tất cả các đỉnh mà không tạo chu trình (Kruskal Algorithm).

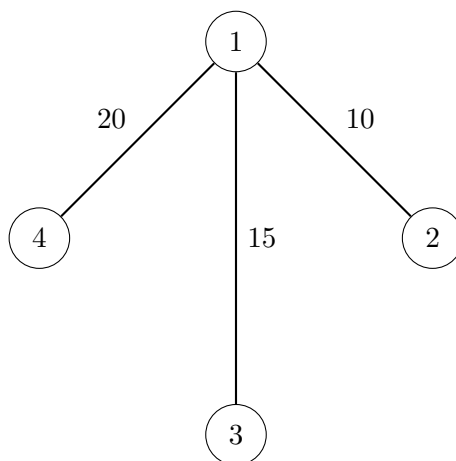
- Chọn cạnh $(1, 2)$ có trọng số 10.
- Chọn cạnh $(1, 3)$ có trọng số 15.
- Chọn cạnh $(1, 4)$ có trọng số 20.

Ba cạnh này đã nối được tất cả 4 đỉnh và không tạo chu trình. Do đó, ta thu được một cây khung nhỏ nhất:

$$T = \{(1, 2), (1, 3), (1, 4)\}.$$

Tổng trọng số của cây là:

$$w(T) = 10 + 15 + 20 = 45.$$



3.5.3 Bước 3. Chọn đỉnh gốc và duyệt theo thứ tự preorder

Chọn đỉnh gốc là thành phố 1.

Cây có dạng: đỉnh 1 nối với các đỉnh 2, 3, 4. Nếu duyệt **preorder** từ đỉnh 1 theo thứ tự các đỉnh con là 2, 3, 4, ta thu được thứ tự thăm lần đầu:

$$1, 2, 3, 4.$$

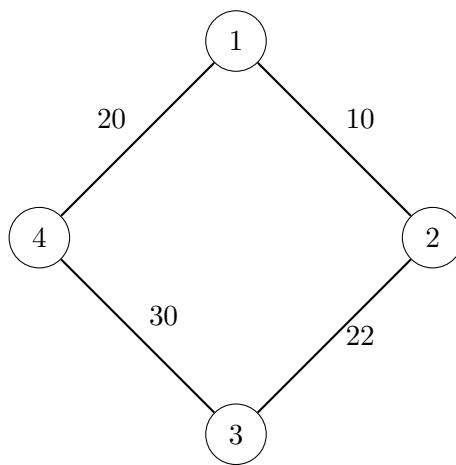
3.5.4 Bước 4. Lập chu trình xấp xỉ

Nối các thành phố theo thứ tự thăm lần đầu và quay lại điểm xuất phát. Khi đó ta được chu trình:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1.$$

Chi phí của hành trình này là:

$$c_{1,2} + c_{2,3} + c_{3,4} + c_{4,1} = 10 + 22 + 30 + 20 = 82.$$



3.5.5 So sánh với nghiệm tối ưu

Trong ví dụ này, nghiệm tối ưu là:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

hoặc

$$1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1,$$

với chi phí tối ưu:

$$OPT = 80.$$

Do đó, nghiệm xấp xỉ ở trên chưa phải là tối ưu, vì: $82 > 80$.

Tuy nhiên, theo lý thuyết thì kết quả xấp xỉ nhỏ hơn $2w(MST)$. Thật vậy, ta có:

$$2w(MST) = 2 \cdot 45 = 90 \geq 82.$$

Ngoài ra, theo lý thuyết của thuật toán xấp xỉ cho metric TSP, ta cũng có:

$$82 < 2 \times OPT = 2 \cdot 80 = 160.$$

Độ phức tạp của thuật toán xấp xỉ này chỉ là **polynomial time**.

4 Bài toán cho thuê máy

4.1 Phát biểu bài toán

Một chủ máy nhận được yêu cầu thuê máy từ nhiều khách hàng. Mỗi khách cần thuê máy vào một số ngày nhất định. Vì chỉ có một chiếc máy, nên nếu chọn hai khách hàng có ngày thuê trùng nhau thì không thể đáp ứng đồng thời.

Ta cần chọn một tập khách hàng sao cho:

- không có sự chồng chéo ngày thuê;
- tổng số ngày cho thuê là lớn nhất.

Mỗi phương án là một tập con các khách hàng. Một phương án chỉ hợp lệ khi các tập ngày thuê của các khách trong đó đôi một không giao nhau.

4.2 Ví dụ

Giả sử có 4 khách hàng:

Khách	Các ngày thuê
1	$\{1, 2\}$
2	$\{3, 4\}$
3	$\{2, 3\}$
4	$\{5\}$

Số ngày thuê của từng khách:

$$|N_1| = 2, \quad |N_2| = 2, \quad |N_3| = 2, \quad |N_4| = 1.$$

4.3 Giải bằng duyệt toàn bộ

Ta xét các tập con của tập khách hàng $\{1, 2, 3, 4\}$.

- $\emptyset$: không chọn khách hàng nào, tổng số ngày = 0.
- $\{1\}$: hợp lệ, tổng số ngày = 2.
- $\{2\}$: hợp lệ, tổng số ngày = 2.
- $\{3\}$: hợp lệ, tổng số ngày = 2.
- $\{4\}$: hợp lệ, tổng số ngày = 1.
- $\{1, 2\}$: $\{1, 2\} \cap \{3, 4\} = \emptyset$, hợp lệ. Tổng số ngày:

$$2 + 2 = 4.$$

- $\{1, 3\}$: $\{1, 2\} \cap \{2, 3\} = \{2\} \neq \emptyset$, nên không hợp lệ.

- $\{1, 4\}$: không giao nhau, hợp lệ. Tổng số ngày:

$$2 + 1 = 3.$$

- $\{2, 3\}$: $\{3, 4\} \cap \{2, 3\} = \{3\} \neq \emptyset$, không hợp lệ.

- $\{2, 4\}$: không giao nhau, hợp lệ. Tổng số ngày:

$$2 + 1 = 3.$$

- $\{3, 4\}$: không giao nhau, hợp lệ. Tổng số ngày:

$$2 + 1 = 3.$$

- $\{1, 2, 4\}$: các tập ngày thuê đôi một không giao nhau, nên hợp lệ. Tổng số ngày:

$$2 + 2 + 1 = 5.$$

- $\{1, 3, 4\}$: không hợp lệ vì 1 và 3 trùng ngày.

- $\{2, 3, 4\}$: không hợp lệ vì 2 và 3 trùng ngày.

- $\{1, 2, 3\}$: không hợp lệ.

- $\{1, 2, 3, 4\}$: không hợp lệ.

Từ đó phương án tối ưu là:

$$J^* = \{1, 2, 4\}$$

với tổng số ngày cho thuê lớn nhất là:

$$f(J^*) = 5.$$

4.3.1 Thuật toán duyệt toàn bộ

Mỗi khách hàng hoặc được chọn, hoặc không được chọn. Nếu có M khách hàng thì có 2^M tập con khách hàng cần xét.

Ta duyệt toàn bộ các tập con:

- kiểm tra xem có trùng ngày thuê hay không;
- nếu không trùng thì tính tổng số ngày cho thuê;
- chọn phương án có tổng số ngày lớn nhất.

Dữ liệu vào:

- M : số khách hàng;
- N_i : tập ngày thuê của khách hàng thứ i .

Dữ liệu ra:

- tập khách hàng tối ưu J^* ;
- tổng số ngày cho thuê lớn nhất.

Thuật toán giả:

Bước 1. Khởi tạo $FOPT = -\infty$, $JOPT = \emptyset$.

Bước 2. Duyệt mọi tập con J của $\{1, 2, \dots, M\}$.

Bước 3. Với mỗi tập con J , kiểm tra xem các khách được chọn có ngày thuê đôi một không giao nhau hay không.

Bước 4. Nếu hợp lệ, tính:

$$S = \text{tổng số ngày thuê của các khách trong } J.$$

Bước 5. Nếu $S > FOPT$ thì cập nhật:

$$FOPT = S, \quad JOPT = J.$$

Bước 6. Trả về $(JOPT, FOPT)$.

Độ phức tạp của cách duyệt toàn bộ là cỡ 2^M vì có 2^M tập con khách hàng cần xét.

4.4 Giải bằng phương pháp nhánh cận

Ví dụ minh họa phương pháp nhánh cận

Xét bài toán với 4 khách hàng:

$$N_1 = \{1, 2\}, \quad N_2 = \{3, 4\}, \quad N_3 = \{2, 3\}, \quad N_4 = \{5\}.$$

Trọng số của mỗi khách hàng chính là số ngày thuê:

$$w_1 = 2, \quad w_2 = 2, \quad w_3 = 2, \quad w_4 = 1.$$

Ta cần chọn một tập khách hàng sao cho các tập ngày thuê đôi một rời nhau và tổng trọng số là lớn nhất.

Hai khách hàng xung đột nếu tập ngày thuê của họ giao nhau. Trong ví dụ này:

$$N_1 \cap N_3 = \{2\} \neq \emptyset, \quad N_2 \cap N_3 = \{3\} \neq \emptyset.$$

Do đó:

- Khách 1 xung đột với khách 3;
- Khách 2 xung đột với khách 3;
- Các cặp còn lại không xung đột.

Ý tưởng nhánh cận

Ta xét lần lượt từng khách hàng theo thứ tự 1, 2, 3, 4. Ở mỗi bước, ta có hai nhánh:

- nhận khách hàng đang xét;
- không nhận khách hàng đang xét.

Ký hiệu:

- S : tập khách hàng đã chọn;
- $f(S)$: tổng trọng số hiện tại;
- UB : cận trên của nút, được tính bằng tổng trọng số hiện tại cộng với tổng trọng số của tất cả khách hàng chưa xét.

Nếu UB không lớn hơn giá trị phương án tốt nhất hiện có, ta cắt nút đó.

Cây nhánh cận

Ban đầu:

$$S = \emptyset, \quad f(S) = 0, \quad UB = 2 + 2 + 2 + 1 = 7.$$

1. Xét khách 1

- **Nhánh 1: nhận khách 1**

Khi đó:

$$S = \{1\}, \quad f(S) = 2.$$

Các khách chưa xét là 2, 3, 4, nên

$$UB = 2 + 2 + 2 + 1 = 7.$$

(a) Xét khách 2

Khách 2 không xung đột với khách 1.

- **Nhánh 1.1: nhận khách 2**

$$S = \{1, 2\}, \quad f(S) = 4.$$

Các khách chưa xét là 3, 4, nên

$$UB = 4 + 2 + 1 = 7.$$

i. Xét khách 3

Khách 3 xung đột với cả khách 1 và khách 2, nên không thể nhận.

* Chỉ còn nhánh loại khách 3:

$$S = \{1, 2\}, \quad f(S) = 4.$$

Xét tiếp chọn khách 4 thì $S = \{1, 2, 4\}$, $f(S) = 5$. Đây là phương án khả thi. Đặt giá trị tốt nhất hiện tại là $z^* = 5$.

Khi không nhận khách 4 thì: $S = \{1, 2\}$, $f(S) = 4$. Kém hơn phương án tốt nhất hiện tại.

– Nhánh 1.2: không nhận khách 2

$$S = \{1\}, \quad f(S) = 2.$$

Các khách chưa xét là 3, 4, nên

$$UB = 2 + 2 + 1 = 5.$$

Vì $UB = 5 = z^*$, nhánh này không thể cho phương án tốt hơn phương án hiện tại, nên có thể cắt.

• Nhánh 2: không nhận khách 1

$$S = \emptyset, \quad f(S) = 0.$$

Các khách chưa xét là 2, 3, 4, nên

$$UB = 2 + 2 + 1 = 5.$$

Vì $UB = 5 = z^*$, nhánh này cũng không thể cho phương án tốt hơn phương án hiện tại, nên cắt.

Từ đó, phương án tối ưu tìm được là:

$$S^* = \{1, 2, 4\},$$

với tổng số ngày cho thuê lớn nhất là

$$f(S^*) = 2 + 2 + 1 = 5.$$

4.4.1 Phương án bộ phận

Giả sử ta xét lần lượt các khách hàng từ 1 đến M . Ở mức k , ta đã quyết định đối với các khách hàng:

$$1, 2, \dots, k.$$

Mỗi khách hàng hoặc được chọn, hoặc không được chọn.

Ký hiệu:

- S_k : tập các khách đã chọn đến mức k ;
- $V_k = \sum_{i \in S_k} |N_i|$: tổng số ngày thuê hiện tại.

4.4.2 Xây dựng cận trên

Vì đây là bài toán cực đại, ta cần một cận trên.

Một cận trên đơn giản là:

$$g_k = V_k + \sum_{i=k+1}^M |N_i|.$$

Cận này giả sử lạc quan rằng ta có thể chọn tất cả các khách còn lại, dù trên thực tế có thể bị trùng ngày. Vì vậy nó là một cận trên hợp lệ.

Có thể dùng cận tốt hơn nếu sắp xếp khách hàng theo số ngày thuê giảm dần hoặc theo quy tắc nào đó, nhưng cận trên đơn giản trên đây là dễ trình bày nhất.

4.4.3 Điều kiện loại nhánh

- Nếu việc thêm khách hàng mới làm trùng ngày thuê với tập đã chọn, nhánh đó không hợp lệ, loại ngay.

- Nếu

$$g_k \leq FOPT,$$

thì dù chọn tốt nhất các khách còn lại, ta cũng không thể vượt nghiệm hiện tại, nên loại.

4.4.4 Thuật toán nhánh cận

Dữ liệu vào:

- M : số khách hàng;
- các tập ngày thuê $N_1, N_2, \dots, N_M$.

Dữ liệu ra:

- tập khách hàng tối ưu J^* ;
- tổng số ngày cho thuê lớn nhất.

Thuật toán:

Bước 1. Khởi tạo:

$$FOPT = 0, \quad JOPT = \emptyset.$$

Bước 2. Tại mức k , xét khách hàng k với hai khả năng:

- chọn khách hàng k ;
- không chọn khách hàng k .

Bước 3. Với mỗi phương án bộ phận:

- kiểm tra điều kiện không trùng ngày;
- tính giá trị hiện tại V_k ;
- tính cận trên

$$g_k = V_k + \sum_{i=k+1}^M |N_i|;$$

- nếu phương án không hợp lệ thì loại;
- nếu $g_k \leq FOPT$ thì loại;
- nếu $k = M$ và $V_k > FOPT$ thì cập nhật nghiệm tốt nhất.

Bước 4. Kết thúc khi không còn nhánh nào cần xét.

4.5 Giải bài toán bằng quy hoạch nguyên 0-1 (Integer Programming)

Xét bài toán với 4 khách hàng:

$$N_1 = \{1, 2\}, \quad N_2 = \{3, 4\}, \quad N_3 = \{2, 3\}, \quad N_4 = \{5\}.$$

Trọng số của mỗi khách hàng là số ngày thuê:

$$w_1 = 2, \quad w_2 = 2, \quad w_3 = 2, \quad w_4 = 1.$$

Mô hình quy hoạch nguyên

Đặt biến quyết định:

$$x_i = \begin{cases} 1, & \text{nếu chấp nhận khách } i, \\ 0, & \text{nếu từ chối khách } i, \end{cases} \quad i = 1, 2, 3, 4.$$

Hàm mục tiêu Tối đa hóa tổng số ngày cho thuê:

$$\max z = 2x_1 + 2x_2 + 2x_3 + x_4.$$

Các ràng buộc Vì mỗi ngày chỉ có một chiếc máy, nên với mỗi ngày không được có quá một khách thuê. Từ đó ta có các ràng buộc xung đột:

$$x_1 + x_3 \leq 1 \quad (\text{ngày 2}),$$

$$x_2 + x_3 \leq 1 \quad (\text{ngày 3}).$$

Các biến là nhị phân:

$$x_1, x_2, x_3, x_4 \in \{0, 1\}.$$

Giải bài toán

Từ các ràng buộc:

$$x_1 + x_3 \leq 1, \quad x_2 + x_3 \leq 1,$$

suy ra khách 3 không thể được chọn đồng thời với khách 1 hoặc khách 2.

Xét hai trường hợp:

Trường hợp 1: $x_3 = 1$ Khi đó bắt buộc:

$$x_1 = 0, \quad x_2 = 0.$$

Ta vẫn có thể chọn khách 4, do đó phương án tốt nhất là:

$$(x_1, x_2, x_3, x_4) = (0, 0, 1, 1),$$

với giá trị:

$$z = 2 \cdot 0 + 2 \cdot 0 + 2 \cdot 1 + 1 \cdot 1 = 3.$$

Trường hợp 2: $x_3 = 0$ Khi đó không còn xung đột giữa các khách 1, 2 và 4, nên ta chọn:

$$x_1 = 1, \quad x_2 = 1, \quad x_4 = 1.$$

Do đó:

$$(x_1, x_2, x_3, x_4) = (1, 1, 0, 1),$$

với giá trị:

$$z = 2 \cdot 1 + 2 \cdot 1 + 2 \cdot 0 + 1 \cdot 1 = 5.$$

Kết luận

Vì $5 > 3$, phương án tối ưu là:

$$(x_1, x_2, x_3, x_4) = (1, 1, 0, 1).$$

Tức là ông chủ nên chấp nhận các khách hàng 1, 2 và 4, và từ chối khách 3. Tổng số ngày cho thuê lớn nhất là:

$$z^* = 5.$$

4.6 Minh hoạ bài toán trong slide bằng mô hình IP

Xét ví dụ có 10 khách hàng và 20 ngày làm việc. Tập ngày thuê của từng khách là:

$$\begin{aligned} N_1 &= \{5, 18\}, & N_2 &= \{2, 4, 12, 14\}, & N_3 &= \{1, 6, 15, 18\}, \\ N_4 &= \{3, 10, 19\}, & N_5 &= \{2, 3, 4, 15\}, & N_6 &= \{7, 16\}, \\ N_7 &= \{6, 19\}, & N_8 &= \{13\}, & N_9 &= \{1, 5, 10\}, \\ N_{10} &= \{12, 20\}. \end{aligned}$$

Mỗi khách hàng i có trọng số $w_i = |N_i|$, tức là:

$$(w_1, \dots, w_{10}) = (2, 4, 4, 3, 4, 2, 2, 1, 3, 2).$$

Mô hình IP. Đặt

$$x_i = \begin{cases} 1, & \text{nếu chọn khách hàng } i, \\ 0, & \text{nếu không chọn khách hàng } i, \end{cases} \quad i = 1, \dots, 10.$$

Ta cần tối đa hoá tổng số ngày cho thuê:

$$\max z = 2x_1 + 4x_2 + 4x_3 + 3x_4 + 4x_5 + 2x_6 + 2x_7 + x_8 + 3x_9 + 2x_{10}.$$

Với mỗi cạnh $(i, j) \in E$, ta thêm ràng buộc:

$$x_i + x_j \leq 1.$$

Cụ thể, mô hình là:

$$\max z = 2x_1 + 4x_2 + 4x_3 + 3x_4 + 4x_5 + 2x_6 + 2x_7 + x_8 + 3x_9 + 2x_{10}$$

với các ràng buộc

$$\begin{aligned}x_1 + x_3 &\leq 1, & x_1 + x_9 &\leq 1, \\x_2 + x_5 &\leq 1, & x_2 + x_{10} &\leq 1, \\x_3 + x_5 &\leq 1, & x_3 + x_7 &\leq 1, & x_3 + x_9 &\leq 1, \\x_4 + x_5 &\leq 1, & x_4 + x_7 &\leq 1, & x_4 + x_9 &\leq 1,\end{aligned}$$

và

$$x_i \in \{0, 1\}, \quad i = 1, \dots, 10.$$

Lời giải tối ưu. Từ kết quả tính toán (sử dụng solver Gurobi ‘import gurobipy as gp, from gurobipy import GRB’), có ít nhất hai phương án tối ưu:

$$(x_1, \dots, x_{10}) = (0, 0, 0, 0, 1, 1, 1, 1, 1, 1),$$

tức là chọn các khách

$$\{5, 6, 7, 8, 9, 10\},$$

với tổng số ngày cho thuê là

$$4 + 2 + 2 + 1 + 3 + 2 = 14.$$

Một phương án tối ưu khác là

$$(x_1, \dots, x_{10}) = (0, 1, 1, 1, 0, 1, 0, 1, 0, 0),$$

tức là chọn các khách

$$\{2, 3, 4, 6, 8\},$$

với tổng số ngày cho thuê là

$$4 + 4 + 3 + 2 + 1 = 14.$$

Do đó, giá trị tối ưu của bài toán là

$$z^* = 14.$$

5 Bài toán phân công công việc

5.1 Phát biểu bài toán

Có N người và N công việc. Mỗi người đều có thể làm mọi công việc, nhưng thời gian thực hiện là khác nhau. Gọi c_{ij} là thời gian để người i làm công việc j .

Cần phân công sao cho:

- mỗi người nhận đúng 1 việc;
- mỗi việc được giao đúng 1 người;
- tổng thời gian là nhỏ nhất.

5.2 Ví dụ

Có 3 người và 3 công việc. Ma trận thời gian là:

$$C = \begin{pmatrix} 9 & 2 & 7 \\ 6 & 4 & 3 \\ 5 & 8 & 1 \end{pmatrix}$$

- người 1 làm việc 1 mất 9 đơn vị thời gian, làm việc 2 mất 2, làm việc 3 mất 7;
- người 2 làm việc 1 mất 6, việc 2 mất 4, việc 3 mất 3;
- người 3 làm việc 1 mất 5, việc 2 mất 8, việc 3 mất 1.

5.3 Giải bằng duyệt toàn bộ

Ta xét tất cả các hoán vị của $(1, 2, 3)$.

1. $(1, 2, 3)$: Người 1 làm việc 1, người 2 làm việc 2, người 3 làm việc 3.

$$9 + 4 + 1 = 14$$

2. $(1, 3, 2)$: Người 1 làm việc 1, người 2 làm việc 3, người 3 làm việc 2.

$$9 + 3 + 8 = 20$$

3. $(2, 1, 3)$: Người 1 làm việc 2, người 2 làm việc 1, người 3 làm việc 3.

$$2 + 6 + 1 = 9$$

4. $(2, 3, 1)$: Người 1 làm việc 2, người 2 làm việc 3, người 3 làm việc 1.

$$2 + 3 + 5 = 10$$

5. $(3, 1, 2)$: Người 1 làm việc 3, người 2 làm việc 1, người 3 làm việc 2.

$$7 + 6 + 8 = 21$$

6. $(3, 2, 1)$: Người 1 làm việc 3, người 2 làm việc 2, người 3 làm việc 1.

$$7 + 4 + 5 = 16$$

Từ đó, phương án tối ưu là:

$$(2, 1, 3)$$

Tổng thời gian nhỏ nhất là: $f^* = 9$.

5.3.1 Thuật toán duyệt toàn bộ

Một phương án phân công tương ứng với một hoán vị $(x_1, x_2, \dots, x_N)$, trong đó $x_i = j$ nghĩa là người i được giao công việc j .

Ta duyệt toàn bộ các hoán vị, tính tổng chi phí của từng phương án, rồi chọn phương án có tổng chi phí nhỏ nhất.

Dữ liệu vào:

- N : số người và số công việc;
- ma trận chi phí $C[N][N]$.

Dữ liệu ra:

- phương án phân công tối ưu;
- tổng chi phí nhỏ nhất.

Thuật toán giả:

Bước 1. Khởi tạo $FOPT = +\infty$, $XOPT = \emptyset$.

Bước 2. Duyệt mọi hoán vị $X = (x_1, x_2, \dots, x_N)$ của $\{1, 2, \dots, N\}$.

Bước 3. Với mỗi hoán vị X , tính:

$$S = c_{1,x_1} + c_{2,x_2} + \dots + c_{N,x_N}.$$

Bước 4. Nếu $S < FOPT$ thì cập nhật:

$$FOPT = S, \quad XOPT = X.$$

Bước 5. Trả về $(XOPT, FOPT)$.

Độ phức tạp của cách duyệt toàn bộ là cỡ $N!$ vì cần xét mọi hoán vị của N công việc.

5.4 Bài toán phân công công việc bằng phương pháp nhánh cận

5.4.1 Phương án bộ phận

Ở mức k , ta đã phân công công việc cho k người đầu tiên:

$$x_1, x_2, \dots, x_k.$$

Ký hiệu chi phí hiện tại:

$$\delta_k = \sum_{i=1}^k c_{i,x_i}.$$

5.4.2 Xây dựng cận dưới

Ta cần ước lượng chi phí nhỏ nhất có thể thêm vào cho các người còn lại.

Một cận dưới đơn giản là:

$$g(x_1, \dots, x_k) = \delta_k + \sum_{i=k+1}^N m_i,$$

trong đó

$$m_i = \min_{1 \leq j \leq N} c_{ij}.$$

Nghĩa là với mỗi người chưa phân công, ta cộng thêm chi phí nhỏ nhất mà người đó có thể đạt được. Vì thực tế các công việc còn phải khác nhau, nên tổng chi phí thật sự sẽ không nhỏ hơn giá trị này. Do đó đây là một cận dưới hợp lệ.

5.4.3 Điều kiện loại nhánh

Vì đây là bài toán cực tiểu, nếu

$$g(x_1, \dots, x_k) \geq FOPT,$$

thì nhánh đó không thể cho phương án tốt hơn phương án tốt nhất hiện tại, nên loại.

5.4.4 Thuật toán nhánh cận

Dữ liệu vào:

- N : số người và số công việc;
- ma trận chi phí $C[N][N]$.

Dữ liệu ra:

- phương án phân công tối ưu;
- tổng chi phí nhỏ nhất.

Thuật toán:

Bước 1. Khởi tạo:

$$FOPT = +\infty, \quad XOPT = \emptyset.$$

Bước 2. Tại mức k , chọn một công việc chưa dùng để gán cho người k .

Bước 3. Với mỗi phương án bộ phận:

- tính chi phí hiện tại

$$\delta_k = \sum_{i=1}^k c_{i,x_i};$$

- tính cận dưới

$$g(x_1, \dots, x_k) = \delta_k + \sum_{i=k+1}^N m_i,$$

trong đó $m_i = \min_j c_{ij}$;

- nếu $g(x_1, \dots, x_k) \geq FOPT$ thì loại;
- nếu $k = N$ và $\delta_k < FOPT$ thì cập nhật nghiệm tốt nhất.

Bước 4. Kết thúc khi không còn nhánh nào cần xét.

5.4.5 Giải ví dụ bằng phương pháp nhánh cận

Xét bài toán phân công với ma trận chi phí

$$C = \begin{pmatrix} 9 & 2 & 7 \\ 6 & 4 & 3 \\ 5 & 8 & 1 \end{pmatrix}.$$

Trong đó, c_{ij} là chi phí để người i thực hiện công việc j . Ta cần phân công mỗi người đúng một công việc, mỗi công việc được giao cho đúng một người, sao cho tổng chi phí nhỏ nhất.

Ta phân nhánh theo từng người. Ở mỗi bước, ta chọn một công việc chưa được giao cho người đang xét. Tại mỗi nút, ta tính:

- g : chi phí đã gán;
- LB : cận dưới = chi phí đã gán cộng với tổng chi phí nhỏ nhất có thể đạt được cho các người còn lại trên các công việc chưa sử dụng.

Nếu LB không nhỏ hơn nghiệm tốt nhất hiện có, ta cắt nhánh đó.

Bước 1: Tìm nghiệm khả thi ban đầu

Xét phương án:

$$(1 \rightarrow 2), \quad (2 \rightarrow 1), \quad (3 \rightarrow 3).$$

Tổng chi phí là:

$$2 + 6 + 1 = 9.$$

Do đó, ta có cận trên ban đầu:

$$z^* = 9.$$

Bước 2: Phân nhánh theo người 1

Nhánh 1: Gán người 1 làm việc 1 Chi phí đã gán:

$$g = 9.$$

Còn lại người 2, người 3 và các công việc 2, 3.

Chi phí nhỏ nhất có thể cho phần còn lại là:

$$\min\{4, 3\} + \min\{8, 1\} = 3 + 1 = 4.$$

Suy ra cận dưới:

$$LB = 9 + 4 = 13.$$

Vì

$$13 > 9 = z^*,$$

nên cắt nhánh này.

Nhánh 2: Gán người 1 làm việc 2 Chi phí đã gán:

$$g = 2.$$

Còn lại người 2, người 3 và các công việc 1, 3.

Chi phí nhỏ nhất có thể cho phần còn lại là:

$$\min\{6, 3\} + \min\{5, 1\} = 3 + 1 = 4.$$

Do đó:

$$LB = 2 + 4 = 6 < 9.$$

Nhánh này còn triển vọng, nên tiếp tục phân nhánh.

Nhánh 2.1: Gán người 2 làm việc 1 Khi đó người 3 phải làm việc 3. Tổng chi phí là:

$$2 + 6 + 1 = 9.$$

Đây là một nghiệm khả thi với giá trị:

$$z = 9.$$

Nhánh 2.2: Gán người 2 làm việc 3 Khi đó người 3 phải làm việc 1. Tổng chi phí là:

$$2 + 3 + 5 = 10.$$

Do đó nhánh này không cải thiện được nghiệm tốt nhất hiện có.

Nhánh 3: Gán người 1 làm việc 3 Chi phí đã gán:

$$g = 7.$$

Còn lại người 2, người 3 và các công việc 1, 2.

Chi phí nhỏ nhất có thể cho phần còn lại là:

$$\min\{6, 4\} + \min\{5, 8\} = 4 + 5 = 9.$$

Suy ra:

$$LB = 7 + 9 = 16.$$

Vì

$$16 > 9 = z^*,$$

nên cắt nhánh này.

Kết thúc, không còn nhánh nào có thể cho giá trị nhỏ hơn 9, nên phương án tối ưu là:

$$(1 \rightarrow 2), \quad (2 \rightarrow 1), \quad (3 \rightarrow 3).$$

Tổng chi phí nhỏ nhất là:

$$z^* = 9.$$

5.4.6 Giải ví dụ bằng Hungarian algorithm

Xét bài toán phân công với ma trận chi phí

$$C = \begin{pmatrix} 9 & 2 & 7 \\ 6 & 4 & 3 \\ 5 & 8 & 1 \end{pmatrix}.$$

Mục tiêu là phân công mỗi người đúng một công việc và mỗi công việc được giao đúng một người sao cho tổng chi phí nhỏ nhất.

Bước 1: Trừ giá trị nhỏ nhất của từng hàng

Giá trị nhỏ nhất của các hàng lần lượt là:

$$\min(9, 2, 7) = 2, \quad \min(6, 4, 3) = 3, \quad \min(5, 8, 1) = 1.$$

Sau khi trừ theo hàng, ta được ma trận:

$$\begin{pmatrix} 9-2 & 2-2 & 7-2 \\ 6-3 & 4-3 & 3-3 \\ 5-1 & 8-1 & 1-1 \end{pmatrix} = \begin{pmatrix} 7 & 0 & 5 \\ 3 & 1 & 0 \\ 4 & 7 & 0 \end{pmatrix}.$$

Bước 2: Trừ giá trị nhỏ nhất của từng cột

Xét các cột của ma trận trên, ta có:

$$\min(7, 3, 4) = 3, \quad \min(0, 1, 7) = 0, \quad \min(5, 0, 0) = 0.$$

Sau khi trừ theo cột, ta được:

$$\begin{pmatrix} 7-3 & 0-0 & 5-0 \\ 3-3 & 1-0 & 0-0 \\ 4-3 & 7-0 & 0-0 \end{pmatrix} = \begin{pmatrix} 4 & 0 & 5 \\ 0 & 1 & 0 \\ 1 & 7 & 0 \end{pmatrix}.$$

Bước 3: Chọn các số 0 độc lập

Ta cần chọn 3 số 0 sao cho không có hai số nào cùng hàng hoặc cùng cột.

Từ ma trận trên:

$$\begin{pmatrix} 4 & 0 & 5 \\ 0 & 1 & 0 \\ 1 & 7 & 0 \end{pmatrix},$$

ta thấy:

- Hàng 1 chỉ có một số 0 tại vị trí $(1, 2)$, nên phải chọn $(1, 2)$.
- Hàng 3 chỉ có một số 0 tại vị trí $(3, 3)$, nên phải chọn $(3, 3)$.
- Khi đó, ở hàng 2 ta phải chọn $(2, 1)$.

Vì vậy, phương án phân công là:

$$(1 \rightarrow 2), \quad (2 \rightarrow 1), \quad (3 \rightarrow 3).$$

Bước 4: Tính chi phí tối ưu

Tra lại ma trận chi phí ban đầu:

$$c_{12} = 2, \quad c_{21} = 6, \quad c_{33} = 1.$$

Do đó tổng chi phí là:

$$2 + 6 + 1 = 9.$$

Từ đó, phương án tối ưu là:

$$(1 \rightarrow 2), \quad (2 \rightarrow 1), \quad (3 \rightarrow 3),$$

với tổng chi phí nhỏ nhất:

$$z^* = 9.$$

Trong ví dụ này, ta đã chọn được đủ 3 số 0 độc lập nên không cần thực hiện thêm bước phủ các số 0 bằng các đường thẳng (**hiệu chỉnh ma trận**).

5.4.7 Giải ví dụ trong slide bằng Hungarian algorithm

Xét bài toán phân công với ma trận chi phí

$$C = \begin{pmatrix} 3 & 1 & 5 & 6 & 1 & 6 & 5 & 4 & 2 & 1 \\ 7 & 8 & 6 & 1 & 9 & 4 & 9 & 2 & 1 & 2 \\ 9 & 3 & 3 & 2 & 1 & 6 & 4 & 1 & 6 & 8 \\ 1 & 2 & 8 & 3 & 6 & 3 & 4 & 6 & 1 & 7 \\ 6 & 2 & 1 & 5 & 6 & 8 & 5 & 5 & 3 & 1 \\ 4 & 7 & 1 & 8 & 9 & 1 & 4 & 5 & 5 & 3 \\ 7 & 6 & 7 & 3 & 1 & 5 & 5 & 5 & 6 & 1 \\ 1 & 7 & 3 & 8 & 2 & 7 & 1 & 4 & 2 & 6 \\ 8 & 8 & 8 & 8 & 1 & 5 & 6 & 8 & 1 & 7 \\ 4 & 1 & 3 & 8 & 5 & 5 & 5 & 1 & 6 & 9 \end{pmatrix}.$$

Bước 1: Trừ giá trị nhỏ nhất của từng hàng

Ta nhận thấy giá trị nhỏ nhất của mỗi hàng đều bằng 1. Vì vậy sau khi trừ 1 cho từng hàng, ta được

$$C' = \begin{pmatrix} 2 & 0 & 4 & 5 & 0 & 5 & 4 & 3 & 1 & 0 \\ 6 & 7 & 5 & 0 & 8 & 3 & 8 & 1 & 0 & 1 \\ 8 & 2 & 2 & 1 & 0 & 5 & 3 & 0 & 5 & 7 \\ 0 & 1 & 7 & 2 & 5 & 2 & 3 & 5 & 0 & 6 \\ 5 & 1 & 0 & 4 & 5 & 7 & 4 & 4 & 2 & 0 \\ 3 & 6 & 0 & 7 & 8 & 0 & 3 & 4 & 4 & 2 \\ 6 & 5 & 6 & 2 & 0 & 4 & 4 & 4 & 5 & 0 \\ 0 & 6 & 2 & 7 & 1 & 6 & 0 & 3 & 1 & 5 \\ 7 & 7 & 7 & 7 & 0 & 4 & 5 & 7 & 0 & 6 \\ 3 & 0 & 2 & 7 & 4 & 4 & 4 & 0 & 5 & 8 \end{pmatrix}.$$

Bước 2: Trừ giá trị nhỏ nhất của từng cột

Với ma trận C' , mỗi cột đều đã có ít nhất một số 0, nên giá trị nhỏ nhất của mỗi cột đều bằng 0. Vì vậy ma trận không thay đổi sau bước này.

Bước 3: Chọn các số 0 độc lập

Ta cần chọn 10 số 0 sao cho mỗi hàng và mỗi cột đúng một số. Một cách chọn là:

$$(1, 2), (2, 4), (3, 5), (4, 1), (5, 3), (6, 6), (7, 10), (8, 7), (9, 9), (10, 8).$$

Tương ứng với phương án phân công:

$$(1 \rightarrow 2), (2 \rightarrow 4), (3 \rightarrow 5), (4 \rightarrow 1), (5 \rightarrow 3), (6 \rightarrow 6), (7 \rightarrow 10), (8 \rightarrow 7), (9 \rightarrow 9), (10 \rightarrow 8).$$

Vì đã chọn được đủ 10 số 0 độc lập, thuật toán Hungarian dừng tại đây.

Bước 4: Tính chi phí tối ưu

Tra lại ma trận chi phí ban đầu:

$$\begin{aligned}c_{12} = 1, \quad c_{24} = 1, \quad c_{35} = 1, \quad c_{41} = 1, \quad c_{53} = 1, \quad c_{66} = 1, \\c_{7,10} = 1, \quad c_{87} = 1, \quad c_{99} = 1, \quad c_{10,8} = 1.\end{aligned}$$

Do đó tổng chi phí là

$$1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 10.$$

Vậy phương án tối ưu là

$(1 \rightarrow 2), (2 \rightarrow 4), (3 \rightarrow 5), (4 \rightarrow 1), (5 \rightarrow 3), (6 \rightarrow 6), (7 \rightarrow 10), (8 \rightarrow 7), (9 \rightarrow 9), (10 \rightarrow 8),$

với tổng chi phí nhỏ nhất

$$z^* = 10.$$

Trong ví dụ này, sau bước trừ theo hàng và cột, ta đã tìm được ngay đủ 10 số 0 độc lập nên không cần thực hiện bước phủ các số 0 bằng các đường thẳng (**hiệu chỉnh ma trận**).

6 So sánh ngắn gọn 4 bài toán

Bài toán	Phương án	Mục tiêu	Ràng buộc
Cái túi	Chọn hoặc không chọn từng đồ vật	Tối đa tổng giá trị	Tổng trọng lượng không vượt quá sức chứa
Người du lịch	Một hoán vị các thành phố	Tối thiểu tổng chi phí hành trình	Đi qua mỗi thành phố đúng 1 lần và quay lại điểm đầu
Cho thuê máy	Một tập con khách hàng	Tối đa tổng số ngày cho thuê	Các ngày thuê không được trùng nhau
Phân công công việc	Một hoán vị công việc giao cho người	Tối thiểu tổng thời gian	Mỗi người 1 việc, mỗi việc 1 người

Với cả 4 bài toán, phương pháp nhánh cận đều tuân theo cùng một khung:

1. xây dựng phương án bộ phận;
2. tính giá trị hiện tại;
3. xây dựng cận;
4. dùng cận để loại bỏ nhánh không cần xét;
5. cập nhật nghiệm tốt nhất khi gặp nghiệm đầy đủ.

Điểm khác nhau chủ yếu nằm ở cách xây dựng cận:

- bài toán cái túi: dùng cận trên từ ý tưởng nhét thêm theo tỉ lệ giá trị/trọng lượng;
- bài toán người du lịch: dùng cận dưới dựa trên cung đường rẻ nhất;
- bài toán cho thuê máy: dùng cận trên từ tổng số ngày của các khách còn lại;
- bài toán phân công công việc: dùng cận dưới từ tổng chi phí nhỏ nhất của các người chưa được phân công.

7 Kết luận chung

Bốn bài toán trên đều là các bài toán tối ưu tổ hợp quen thuộc. Điểm chung là:

- đều có tập phương án hữu hạn;
- đều cần chọn ra phương án tốt nhất theo một tiêu chí;
- nếu giải bằng duyệt toàn bộ thì dễ hiểu nhưng tốn thời gian khi dữ liệu lớn.

Vì vậy, trong thực tế, ngoài phương pháp vét cận, người ta còn dùng nhiều kỹ thuật hiệu quả hơn như:

- quay lui (backtracking);
- nhánh cận (branch-and-bound);
- quy hoạch động (dynamic programming);
- thuật toán tham lam (greedy algorithm);
- các thuật toán xấp xỉ hoặc heuristic (approximation algorithm).

Trong khuôn khổ Toán rời rạc 1, mục tiêu chính không phải là cài đặt thuật toán tối ưu mạnh nhất, mà là hiểu cách mô hình hoá bài toán, thiết kế không gian nghiệm, và dùng cận để cắt tỉa quá trình tìm kiếm.